Exam Rank 05 — Step-by-step pass guide

Oct 29, 2026 · @Bruno

How the exam works

You pass by clearing **one exercise from level 0** (vect2, bigint or polyset, C++) and **one from level 1** (life or bsq, C). The grader compiles your files with its own main and compares the output, so only the subject's behaviour matters. Every solution below was compiled with `-Wall -Wextra -Werror -std=c++98` (C++) or `-Wall -Wextra -Werror` (C) on gcc and clang, checked for leaks with valgrind, and diffed against both reference repos.

Level	Exercise	Language	Files you hand in	Core idea
0	vect2	C++98	vect2.hpp, vect2.cpp	Operator overloading on 2 ints
0	bigint	C++98	bigint.hpp, bigint.cpp	Number stored as a string of digits
0	polyset	C++98	searchable_array_bag.hpp/.cpp, searchable_tree_bag.hpp/.cpp, set.hpp/.cpp	Multiple inheritance + a wrapper
1	life	C	`*.c`, `*.h` optional	Pen drawing + Conway rules B3/S23
1	bsq	C	`*.c`, `*.h` optional	Map validation + dynamic programming

Workflow for every exercise:

1. Read the subject and the given main. List every operator or behaviour the main uses.
2. Write the header first, then the .cpp/.c.
3. Compile with the exam flags: `c++ -Wall -Wextra -Werror -std=c++98 *.cpp` or `cc -Wall -Wextra -Werror *.c`.

4. Run the subject's examples and compare with the expected output (`| cat -e` shows trailing spaces).
5. Only then `git add` , `git commit` , `git push` and ask for grading.

C++98 traps: no `std::to_string` , no `nullptr` , no `auto` . Use `std::ostringstream` to turn a number into a string.

Level 0 · vect2

vect2 passes with one class of 2 ints where every operator is built from `+=` , `-=` and `*=` . The whole class fits inline in the header; the .cpp only holds the two free functions.

1. Store the components as `int v[2]` : `operator[]` becomes `return v[i];` with no if.
2. One constructor with defaults `vect2(int x = 0, int y = 0)` covers `vect2 v1;` and `vect2 v2(1, 2);` . Copy constructor, `=` and destructor are the compiler's.
3. Write `[]` twice: `int &` for `v1[1] = 12` and a `const` version for `v3[1]` on a const object.
4. Write the compound operators `+=` , `-=` , `*=` . They modify and return `*this` by reference, so `v2 += v2 += v3` chains.
5. Build `+` , `-` , `*` from them: `return vect2(*this) += o;` (copy, modify the copy, return it).
6. Unary `-` returns `vect2(-v[0], -v[1])` .
7. `++x` adds `vect2(1, 1)` and returns `*this` ; `x++` copies, calls `++*this` , returns the copy. Same for `--` .
8. `==` compares both, `!=` is `!(*this == o)` .
9. In the .cpp: `int * vect2` (for `3 * v2`) calls `v * n` , and `<<` prints `{x, y}` .

```
// vect2.hpp
#pragma once
#include <iostream>

class vect2
{
        int v[2];

public:
        vect2(int x = 0, int y = 0) { v[0] = x; v[1] = y; }
```

```
        int &operator[](int i) { return v[i]; }
        int operator[](int i) const { return v[i]; }

        vect2 &operator+=(const vect2 &o) { v[0] += o.v[0]; v[1] += o.v[1];
return *this; }
        vect2 &operator-=(const vect2 &o) { v[0] -= o.v[0]; v[1] -= o.v[1];
return *this; }
        vect2 &operator*=(int n) { v[0] *= n; v[1] *= n; return *this; }

        vect2 operator+(const vect2 &o) const { return vect2(*this) += o; }
        vect2 operator-(const vect2 &o) const { return vect2(*this) -= o; }
        vect2 operator*(int n) const { return vect2(*this) *= n; }
        vect2 operator-() const { return vect2(-v[0], -v[1]); }

        vect2 &operator++() { return *this += vect2(1, 1); }
        vect2 &operator--() { return *this -= vect2(1, 1); }
        vect2 operator++(int) { vect2 t(*this); ++*this; return t; }
        vect2 operator--(int) { vect2 t(*this); --*this; return t; }

        bool operator==(const vect2 &o) const { return v[0] == o.v[0] && v[1]
== o.v[1]; }
        bool operator!=(const vect2 &o) const { return !(*this == o); }
};

vect2 operator*(int n, const vect2 &v);
std::ostream &operator<<(std::ostream &os, const vect2 &v);
```

```
// vect2.cpp
#include "vect2.hpp"

vect2 operator*(int n, const vect2 &v)
{
        return v * n;
}

std::ostream &operator<<(std::ostream &os, const vect2 &v)
{
        return os << "{" << v[0] << ", " << v[1] << "}";
}
```

Check: the subject main must print `v1: {-84, -168}`, `v2: {20, 40}` and `-v2: {-20, -40}`. The comments in the main (`// 2, 3`) are wrong; the real `v4++` line prints `{1, 2}`.

Level 0 · bigint

bigint passes with one `std::string` of digits, one addition loop and shifts that take a `bigint`. Because the constructor from `unsigned long` is implicit, `b << 10`, `d <<= 4` and `d >>= (const bigint)2` all hit the same overload, so you write each shift once instead of twice.

1. Member: `std::string s;` holding the digits, most significant first ("1337").
2. Constructor `bigint(unsigned long n = 0)`: write `n` into an `ostringstream` and take `.str()`. It also serves as the default constructor (`bigint c;` gives "0").
3. `+=`: walk both strings from the right with a carry `c`; add the digits into `c`, insert `'0' + c % 10` at the front, keep `c /= 10`. Loop while digits or carry remain.
4. `+` = `return bigint(*this) += o;`. `++x` = `*this += 1`. `x++` = copy, `++*this`, return the copy.
5. `<<=`: append that many `'0'`, but not to "0" (0 << 5 must stay 0).
6. `>>=`: cut that many digits from the right; cutting all of them gives "0".
7. Shift amount: read the other bigint's string back into an `unsigned long` with an `istringstream` (`to_ul`).
8. `<`: shorter string = smaller; same length = compare the strings. Write `>`, `<=`, `>=` from `<`, and `==` / `!=` on the strings.
9. `<<` to ostream prints the string. No leading zeros ever appear, because nothing creates them.

```
// bigint.hpp
#pragma once
#include <iostream>
#include <sstream>
#include <string>

class bigint
{
        std::string s;

public:
        bigint(unsigned long n = 0) { std::ostringstream o; o << n; s =
o.str(); }

        bigint &operator+=(const bigint &o);
        bigint operator+(const bigint &o) const { return bigint(*this) += o;
```

```
}
        bigint &operator++() { return *this += 1; }
        bigint operator++(int) { bigint t(*this); ++*this; return t; }

        bigint &operator<<=(const bigint &n);
        bigint &operator>>=(const bigint &n);
        bigint operator<<(const bigint &n) const { return bigint(*this) <<=
n; }
        bigint operator>>(const bigint &n) const { return bigint(*this) >>=
n; }

        bool operator<(const bigint &o) const
        {
                return s.size() != o.s.size() ? s.size() < o.s.size() : s <
o.s;
        }
        bool operator>(const bigint &o) const { return o < *this; }
        bool operator<=(const bigint &o) const { return !(o < *this); }
        bool operator>=(const bigint &o) const { return !(*this < o); }
        bool operator==(const bigint &o) const { return s == o.s; }
        bool operator!=(const bigint &o) const { return s != o.s; }

        const std::string &str() const { return s; }
};

std::ostream &operator<<(std::ostream &os, const bigint &b);
```

```
// bigint.cpp
#include "bigint.hpp"

static unsigned long to_ul(const std::string &s)
{
        unsigned long n = 0;
        std::istringstream(s) >> n;
        return n;
}

bigint &bigint::operator+=(const bigint &o)
{
        std::string r;
        int i = s.size() - 1, j = o.s.size() - 1, c = 0;

        while (i >= 0 || j >= 0 || c)
```

```
        {
                if (i >= 0) c += s[i--] - '0';
                if (j >= 0) c += o.s[j--] - '0';
                r.insert(r.begin(), char('0' + c % 10));
                c /= 10;
        }
        s = r;
        return *this;
}

bigint &bigint::operator<<=(const bigint &n)
{
        if (s != "0")
                s.append(to_ul(n.s), '0');
        return *this;
}

bigint &bigint::operator>>=(const bigint &n)
{
        unsigned long k = to_ul(n.s);
        s = k >= s.size() ? "0" : s.substr(0, s.size() - k);
        return *this;
}

std::ostream &operator<<(std::ostream &os, const bigint &b)
{
        return os << b.str();
}
```

Check: `(b << 10) + 42 = 230000000042`, `(d <<= 4) = 13370000`, `(d >>= 2) = 133700`, and the six comparisons print `0 0 1 1 0 1`.

Do not add `operator unsigned long()` to the class: `(b << 10) + 42` would become ambiguous between bigint `+` and built-in `+`.

Level 0 · polyset

polyset passes with two tiny subclasses that add `has()` and a `set` that checks `has()` before inserting. Do not touch the given files (bag, searchable_bag, array_bag, tree_bag).

1. Read the given headers: `array_bag` has `protected int *data; int size;`, `tree_bag` has `protected node *tree` with `l`, `r`, `value`. You can use them in subclasses.
2. `searchable_array_bag : public array_bag, public searchable_bag`. Both parents inherit `virtual public bag`, so there is only one `bag` inside: no ambiguity.
3. Canonical form in one line each: default ctor `{}`, copy ctor `: array_bag(o) {}`, `operator=` calls `array_bag::operator=(o)`, empty destructor.
4. `has()` for the array: loop `i` from 0 to `size`, return true on a match.
5. Same pattern for `searchable_tree_bag` with `tree_bag`. Its `has()` walks the binary search tree: go left if `n < value`, right otherwise, stop on a match or a null node.
6. `set` keeps a **reference** `searchable_bag &b` to the bag it wraps (the main builds `set sa(*a);` and then calls `sa.get_bag()`).
7. `set::insert(int)` = `if (!b.has(n)) b.insert(n);`. This is the whole point: array_bag accepts duplicates, a set must not.
8. `set::insert(int *, int)` loops over the single insert, so duplicates inside the array are dropped too.
9. `has`, `print`, `clear` forward to the bag; `get_bag()` exists in a const and a non-const version.
10. Declarations go in `set.hpp`, the bodies in `set.cpp`.

```
// searchable_array_bag.hpp
#pragma once
#include "array_bag.hpp"
#include "searchable_bag.hpp"

class searchable_array_bag : public array_bag, public searchable_bag
{
public:
        searchable_array_bag() {}
        searchable_array_bag(const searchable_array_bag &o) : array_bag(o) {}
        searchable_array_bag &operator=(const searchable_array_bag &o) {
array_bag::operator=(o); return *this; }
        ~searchable_array_bag() {}

        bool has(int n) const;
};
```

```
// searchable_array_bag.cpp
#include "searchable_array_bag.hpp"

bool searchable_array_bag::has(int n) const
{
        for (int i = 0; i < size; i++)
                if (data[i] == n)
                        return true;
        return false;
}
```

```
// searchable_tree_bag.hpp
#pragma once
#include "tree_bag.hpp"
#include "searchable_bag.hpp"

class searchable_tree_bag : public tree_bag, public searchable_bag
{
public:
        searchable_tree_bag() {}
        searchable_tree_bag(const searchable_tree_bag &o) : tree_bag(o) {}
        searchable_tree_bag &operator=(const searchable_tree_bag &o) {
tree_bag::operator=(o); return *this; }
        ~searchable_tree_bag() {}

        bool has(int n) const;
};
```

```
// searchable_tree_bag.cpp
#include "searchable_tree_bag.hpp"

bool searchable_tree_bag::has(int n) const
{
        node *cur = tree;

        while (cur && cur->value != n)
                cur = n < cur->value ? cur->l : cur->r;
        return cur != 0;
}
```

```
// set.hpp
#pragma once
#include "searchable_bag.hpp"

class set
{
        searchable_bag &b;

public:
        set(searchable_bag &bag);
        set(const set &o);
        ~set();

        void insert(int n);
        void insert(int *arr, int size);
        bool has(int n) const;
        void print() const;
        void clear();
        searchable_bag &get_bag();
        const searchable_bag &get_bag() const;
};
```

```
// set.cpp
#include "set.hpp"

set::set(searchable_bag &bag) : b(bag) {}
set::set(const set &o) : b(o.b) {}
set::~set() {}

void set::insert(int n)
{
        if (!b.has(n))
                b.insert(n);
}

void set::insert(int *arr, int size)
{
        for (int i = 0; i < size; i++)
                insert(arr[i]);
}

bool set::has(int n) const { return b.has(n); }
```

```
void set::print() const { b.print(); }
void set::clear() { b.clear(); }
searchable_bag &set::get_bag() { return b; }
const searchable_bag &set::get_bag() const { return b; }
```

Check: `./polyset 1 2 3` and `./polyset 5 3 8 3 1 0 -2` give the same output as both reference repos. valgrind shows two leaks, both from the given main (it never deletes `t` and `a`), not from your code.

Compiler note: the 42nice version of the given files uses `nullptr` and a compound literal `(int[]){1, 2, 3, 4}` in main.cpp, so it is C++11 and only clang accepts that main. Compile polyset with `c++ -Wall -Wextra -Werror -std=c++11` (clang); `-std=c++98` only works with the other repo's given files. Your own files above compile either way.

Why no `operator=` in set: a reference member cannot be re-pointed, so the compiler deletes it and the main never uses it. If your grader insists on full canonical form, switch `b` to a pointer `searchable_bag *b;` and add `set() : b(0) {}` plus `set &operator=(const set &o) { b = o.b; return *this; }`, with `b->` instead of `b.`.

Level 1 · life

life passes with a single `main` of about 45 lines: two flat `char` boards, a pen loop over `read`, then the Conway rule applied `iterations` times. It passes all 69 tests of the second repo's suite.

1. Check `ac == 4`, then `w`, `h`, `it` with `atoi`.
2. `calloc` two boards of `w * h` bytes: `b` (current) and `n` (next). Cell (x, y) lives at `b[y * w + x]`; 1 = alive.
3. Pen state: `x = 0, y = 0, pen = 0` (pen starts lifted).
4. Read stdin one char at a time. `w a s d` move the pen, but only if it stays on the board (clamped). `x` toggles the pen.
5. After every character: if the pen is down, set the current cell to 1. This marks the start cell when you press `x` and every cell you move onto.
6. Repeat `it` times: for each cell, count the 8 neighbours that are inside the board; next cell = `k == 3 || (k == 2 && alive)`. Then swap `b` and `n`.
7. Print `h` lines of `w` chars: `'O'` if alive, `' '` if dead, then `'\n'`.
8. Free both boards.

```
// life.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int ac, char **av)
{
	if (ac != 4)
		return 1;
	int w = atoi(av[1]), h = atoi(av[2]), it = atoi(av[3]);
	char *b = calloc(w * h, 1), *n = calloc(w * h, 1), c;
	int x = 0, y = 0, pen = 0;

	while (read(0, &c, 1) > 0)
	{
		if (c == 'w' && y > 0) y--;
		else if (c == 's' && y < h - 1) y++;
		else if (c == 'a' && x > 0) x--;
		else if (c == 'd' && x < w - 1) x++;
		else if (c == 'x') pen = !pen;
		if (pen)
			b[y * w + x] = 1;
	}
	while (it-- > 0)
	{
		for (y = 0; y < h; y++)
			for (x = 0; x < w; x++)
			{
				int k = 0;
				for (int dy = -1; dy <= 1; dy++)
					for (int dx = -1; dx <= 1; dx++)
						if ((dx || dy) && y + dy >= 0
&& y + dy < h && x + dx >= 0 && x + dx < w)
							k += b[(y + dy) * w +
x + dx];
				n[y * w + x] = (k == 3 || (k == 2 && b[y * w
+ x]));
			}
		char *t = b; b = n; n = t;
	}
	for (y = 0; y < h; y++)
	{
		for (x = 0; x < w; x++)
```

```
                        putchar(b[y * w + x] ? 'O' : ' ');
                putchar('\n');
        }
        free(b);
        free(n);
        return 0;
}
```

Check with the subject's examples: `echo 'dxss' | ./life 3 3 1 | cat -e` must print `$`, `OOO$`, `$`.

Border policy: the subject doesn't say what happens when the pen walks off the board. This solution clamps it (stays on the edge), like the first repo; the second repo's suite accepts both policies. If you ever need the other policy, drop the bounds checks in the moves and only draw when `x`, `y` are inside.

Level 1 · bsq

bsq passes with one function per map: validate while reading with `fscanf` + `getline`, then one dynamic-programming pass where each cell stores the side of the biggest empty square ending there. It matched the second repo's solution on 400 random maps and 13 edge-case maps.

1. Header: `fscanf(f, "%d %c %c %c", &n, &e, &o, &full)` must return 4, `n > 0`, and the three chars must differ.
2. The rest of the header line must be exactly `"\n"`: one `getline` that returns 1.
3. Read `n` lines with `getline`. Each must be longer than 1, end with `'\n'`, have the same width `w` as the first, and contain only `e` or `o`.
4. After the `n` lines, another `getline` must hit EOF (extra lines = error).
5. Any failure prints `map error\n` on **stderr**, frees everything and moves on.
6. DP: `d[i][j]` = 0 on an obstacle; on an empty cell, 1 on the first row/column, else `1 + min(up, left, up-left)`.
7. Keep the best with a strict `>`: scanning row by row, left to right, the first square found of a size is the topmost, then leftmost one, which is exactly the tie rule.
8. Fill the square that ends at `(bi, bj)` with `full`, print the lines (they still hold their `'\n'`), free.
9. `main`: no argument = read stdin. Otherwise one map per file, a failed `fopen` = `map error`, and a `"\n"` between two maps.

```
// bsq.c
#include <stdio.h>
#include <stdlib.h>

static int bsq(FILE *f)
{
	int n, w = 0, i, j, ok = 1, best = 0, bi = 0, bj = 0;
	char e, o, full, **m, *line = NULL;
	size_t cap = 0;

	if (fscanf(f, "%d %c %c %c", &n, &e, &o, &full) != 4 || n <= 0
		|| e == o || e == full || o == full
		|| getline(&line, &cap, f) != 1)
		return free(line), fputs("map error\n", stderr), 1;
	m = calloc(n, sizeof(char *));
	for (i = 0; i < n && ok; i++)
	{
		size_t c = 0;
		int len = getline(&m[i], &c, f);
		if (i == 0)
			w = len - 1;
		ok = len > 1 && len - 1 == w && m[i][w] == '\n';
		for (j = 0; ok && j < w; j++)
			ok = m[i][j] == e || m[i][j] == o;
	}
	if (ok && getline(&line, &cap, f) != -1)
		ok = 0;
	int *d = ok ? calloc(n * w, sizeof(int)) : NULL;
	for (i = 0; ok && i < n; i++)
		for (j = 0; j < w; j++)
		{
			int k = 0;
			if (m[i][j] == e)
			{
				k = 1;
				if (i && j)
				{
					k = d[(i - 1) * w + j];
					if (d[i * w + j - 1] < k) k = d[i * w
+ j - 1];
					if (d[(i - 1) * w + j - 1] < k) k =
d[(i - 1) * w + j - 1];
					k++;
```

```
                                }
                        }
                        d[i * w + j] = k;
                        if (k > best)
                                best = k, bi = i, bj = j;
                }
        for (i = bi - best + 1; ok && i <= bi; i++)
                for (j = bj - best + 1; j <= bj; j++)
                        m[i][j] = full;
        for (i = 0; i < n; i++)
        {
                if (ok)
                        fputs(m[i], stdout);
                free(m[i]);
        }
        if (!ok)
                fputs("map error\n", stderr);
        free(m);
        free(d);
        free(line);
        return !ok;
}

int main(int ac, char **av)
{
        if (ac == 1)
                return bsq(stdin);
        for (int i = 1; i < ac; i++)
        {
                FILE *f = fopen(av[i], "r");
                if (!f)
                        fputs("map error\n", stderr);
                else
                        bsq(f), fclose(f);
                if (i < ac - 1)
                        fputs("\n", stdout);
        }
        return 0;
}
```

Check: the subject's 9-line map must give a 7×7 square starting at column 5. Test the errors too: lines of different length, a char not in the header, two identical header chars, fewer lines than announced, a missing last `'\n'`, an empty file.

Before you hit grademe

- [ ] Only the expected files are pushed (no binary, no main.cpp unless asked, no `a.out`).
- [ ] File names match the subject exactly. polyset says `searchable_tree_bg.cpp` in one place; the real name is `searchable_tree_bag.cpp`, matching the given main's include.
- [ ] It compiles with `-Wall -Wextra -Werror` (and `-std=c++98` for C++).
- [ ] Every example of the subject gives the same output under `| cat -e`.
- [ ] C++: every method that the main calls on a `const` object is marked `const`.
- [ ] life: lines are exactly `width` chars, dead cells are spaces, alive cells are a capital `O`.
- [ ] bsq: errors go to stderr, results to stdout, stdin works with no argument.
- [ ] `git add` + `git commit` + `git push`, then `git log` shows the commit.